

Answers of the Questions in Assignment 1

1. Assignment Overview

2. Byte-Pair Encoding (BPE) Tokenizer

2.1 Problem (unicode1): Understanding Unicode

1. `chr(0)` returns `'\x00'`, which represents the character U+0000, which is the NULL character (also called "NUL").
2. The string representation `str.__repr()` shows the code needed to create the string and tell the interpreter its not bare characters, but the string literal is what a human wanna read it, like `'abc'` and `"abc"`.
3. `chr(0)` can't be printed, but if inserted into a string, it will be string literal `'\x00'`

2.2 Problem (unicode2): Unicode Encodings

1. Because using UTF-16/UTF-32 need much more space than UTF-8, especially for ASCII characters. And UTF-16/32 introduce more zero bytes than UTF-8, which may introduce less robustness
2. In UTF-8, there are four byte-classes:
 - 0xxxxxxx -> 1-byte char
 - 110xxxxx -> start of 2-byte char, starts with `'\xc3'`
 - 1110xxxx -> start of 3-byte char. starts with `'\xe3'`
 - 11110xxx -> start of 4-byte char, starts with `'\xf0'`

So if a string contains 2/3/4-byte char, directly decoding it byte-by-byte will raise an error as these byte themselves are incomplete. For example, for this `test_string = "hello! こんにちは! "`, its UTF-8 encoded string is `b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'`.

Specifically,

- こ is `\xe3\x81\x93`
- ん is `\xe3\x82\x93`
- に is `\xe3\x81\xab`
- ち is `\xe3\x81\xa1`
- は is `\xe3\x81\xaf`

When decoding this string byte-by-byte and the function reading up to `\xe3`, it will raise an `UnicodeDecodeError`

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    i = 0
    chars = []
    while i < len(bytestring):
        b0 = bytestring[i] # b0 = 11100011
        # Determine number of bytes
        if b0 < 0x80:
            n = 1
            code = b0
        elif 0xC0 <= b0 < 0xE0:
            n = 2
            code = b0 & 0x1F
        elif 0xE0 <= b0 < 0xF0: # 11100011 10000001 10010011
            n = 3
```

```

        code = b0 & 0x0F # code = 00000011
    elif 0xF0 <= b0 < 0xF8:
        n = 4
        code = b0 & 0x07
    else:
        raise ValueError("Invalid UTF-8 lead byte")

    if i + n > len(bytestring):
        raise ValueError("Truncated UTF-8 sequence")

    # Add continuation bytes
    for j in range(1, n):
        cont = bytestring[i + j] # cont = \x81 1100 0000
        # Every non-leading UTF-8 byte (i.e., bytes after the first one in a multi-
        # byte sequence) must begin with the bit pattern 10xxxxxx.
        if cont & 0xC0 != 0x80:
            raise ValueError("Invalid UTF-8 continuation byte")
        # appends 6 bits to the right of the existing ones.
        code = (code << 6) | (cont & 0x3F)

    chars.append(chr(code))
    i += n
return "".join(chars)

```

3. `b"\xe3\x82"`, because in UTF-8, 2-byte characters must begin with 110xxxxx, and a byte seq starting with 0xe3 is 3-byte UTF-8 character.

2.3 Problem (train_bpe_tinystories) BPE Training on TinyStories

1. On Apple M1 Pro with 8 processes, it takes 150.69 seconds (140.95 seconds with 16 processes). The longest token is 'Ġaccomplishment'. Considering of usage of `bytes_to_unicode`, it's `b'accomplishment'` actually
2. The part of find the token pair which having the most occurency frequencies in the merging process takes the most time.

2.4 Problem (train_bpe_owt) BPE Training on OpenWebText

1. The longest token is
"-----". It makes sense because Users frequently use long strings of dashes to separate sections, create horizontal rules, or format ASCII tables.
2. TinyStories vocab size: 10000
OpenWebText vocab size: 32000
TinyStories avg token length: 5.79
OpenWebText avg token length: 6.34
Character distribution:
TinyStories: {'special': 145, 'alphabetic': 57748, 'numeric': 20}
OpenWebText: {'special': 1334, 'alphabetic': 199755, 'numeric': 1698}
Common tokens: 7319
TinyStories unique tokens: 2681
OpenWebText unique tokens: 24681
Jaccard Similarity: 0.2110

Longest TinyStories tokens:
Ġaccomplishment
Ġdisappointment
Ġresponsibility

Guncomfortable
Gcompassionate

Longest OpenWebText tokens:

[illegible]

âGKâGKâGKâGKâGKâGKâGKâGKâGKâGKâGKâGKâGKâGKâGKâGK

Sample unique TinyStories tokens:

['obody', 'washer', 'Ġwarmed', 'Ġtidied', 'Ġbluebird', 'Ġsneaky', 'ĠPuffy', 'ĠFaye', 'Pip', 'bbers']

Sample unique OpenWebText tokens:

```
[ 'ĞUS', 'gets', 'rath', 'Ğthermal', 'gerald', 'nine', 'Ğviolation', 'Ğweakened',  
'rifice', 'Copyright']
```

2.7 Experiments

1. The TinyStories tokenizer(10K vocab) achieves 4.0445 byte/token compression ratio, The OpenWebText tokenizer (32K vocab) achieves 4.5125 byte/token compression ratio. It shows that the larger vocabulary is, the better compression it achieves
2. Using tokenizer trained on TinyStories to encode OpenWebText achieves 3.4111 byte/token compression ratio. Compared with 4.5125 byte/token achieved by native OpenWebText tokenizer, it shows nearly 25% degradation.
3. For single process and treating the document as a whole text chunk, it can achieve 1.32MB/s
4. uint16 is appropriate because it can represent values up to 65,535, which easily accommodates our vocabulary sizes (10K and 32K tokens), while being twice as memory-efficient as uint32 (1.9 MB vs 3.8 MB per 1M tokens) and avoiding the 255 value limit of uint8.

3. Transformer Language Model Architecture

3.6 Problem (transformer_accounting): Transformer LM resource accounting

For given:

```
vocab_size : 50,257
context_length : 1,024
num_layers : 48
d_model : 1,600
num_heads : 25
d_ff : 6,400
```

AB requires $2mnp$ FLOPs

How many trainable parameters would our model have?

Embedding: Each has $\text{num_embeddings}(\text{vocab_size}) \times \text{embedding_dim}(\text{d_model})$ weights

TransformerBlock:

- $\ln1(\text{Norm1})$ has d_{model} weights
- attn1 has $d_{\text{model}} \times d_{\text{model}} \times 4$ weights (q, k, v, o are both $d_{\text{model}} \times d_{\text{model}}$ Linear layers)
- $\text{ffn}(\text{SwiGLUFFN})$ has $d_{\text{model}} \times d_{\text{ff}}$ weights and multiplies with $3(w_1 w_2 w_3)$.
- $\ln2(\text{Norm2})$ has also d_{model} weights

So each TransformerBlock have

$$2 \times d_{\text{model}} + d_{\text{model}} \times d_{\text{model}} \times 4 \text{weights} + d_{\text{model}} \times d_{\text{ff}} \text{weights} \times 3$$

weights We have num_layers of TransformerBlocks

- **Final Norm Layer** is the same, has d_{model} weights
- **Linear Projection Layer** has $d_{\text{model}} \times \text{vocab_size}$ weights

So totally we have 2,127,057,600 parameters

The model would have approximately 2.13 billion trainable parameters, which will take 7.92GB of memory approximately

How many FLOPs do these matrix multiplies require in total?

- Per **q_proj operation** has $2 \times \text{seq_len} \times d_{\text{model}} \times d_{\text{model}}$ FLOPs
- Per **k_proj operation** has $2 \times \text{seq_len} \times d_{\text{model}} \times d_{\text{model}}$ FLOPs
- Per **v_proj operation** has $2 \times \text{seq_len} \times d_{\text{model}} \times d_{\text{model}}$ FLOPs
- Per **o_proj operation** has $2 \times \text{seq_len} \times d_{\text{model}} \times d_{\text{model}}$ FLOPs

Which adding up to 20,971,520,000

- Per **Attention Calculation** has

$$2 \times \text{seq_len} \times d_k \times \text{seq_len} \times \text{num_heads} + 2 \times \text{seq_len} \times d_v \times \text{seq_len} \times \text{num_heads} \times 2$$

FLOPs. In other words, it has $4 \times \text{seq_len} \times \text{seq_len} \times d_{\text{model}}$ FLOPs.

- Per **FFN(SwiGLUFFN)** has: $3 \times 2 \times d_{\text{ff}} \times d_{\text{model}} \times \text{seq_len}$ FLOPs.
- **Final Linear Projection** has $2 \times d_{\text{model}} \times \text{vocab_size} \times \text{seq_len}$ FLOPs

So, the GPT-2 XL-shaped model has 4,513,336,524,800 FLOPs in total, approximately 4.51 TFLOPs

Based on your analysis above, which parts of the model require the most FLOPs?

The SwiGLU ffn takes the most. The heavy lifting is done by the neurons processing information d_{model} to d_{ff} and back, rather than the token-to-token mixing.

As the model size increases, which parts of the Transformer LM take up proportionally more or less of the total FLOPs

As the model gets deeper and wider, FFN & Attention Projections increase proportionally.

FLOPs Distribution Across Model Sizes

Component	Small (12L, 768d)	Medium (24L, 1024d)	Large (36L, 1280d)
FFN Layers	49.8%	59.9%	64.2%
Attn Projections	16.6%	20.0%	21.4%
Attn Calculation	11.1%	10.0%	8.6%
Final Linear Proj	22.6%	10.2%	5.8%
Total FLOPs	350B	1.03T	2.26T

Take GPT-2 XL and increase the context length to 16,384. How does the total FLOPs for one forward pass change? How do the relative contribution of FLOPs of the model components change?

FLOPs Distribution Across Context Lengths

Component	Share (1k Context)	Share (16k Context)	Scaling Behavior
Attention Calculation	7.1%	55.2%	Quadratic (N^2)
FFN Layers	66.9%	32.3%	Linear (N)

Attention Projections	22.3%	10.8%	Linear (N)
Final Projection	3.6%	1.8%	Linear (N)

4. Training a Transformer Language Model

4.2 Problem (learning_rate_tuning) Tuning the Learning Rate

The loss in all of these learning rates are decaying, but the loss decays faster when learning rate is $1e-3$